

数据库参考往年试题五套详细解析

一、2015-2016 学年第 2 学期 2014 级《数据库原理》考试试题 A 解析

第一部分：简答题

1. 原题：数据库管理系统中 DDL 所能完成的操作包括哪些？

参考答案：

DDL 是数据定义语言，主要用于定义和维护数据库对象。常见操作包括：

- 创建对象：CREATE DATABASE、CREATE TABLE、CREATE VIEW、CREATE INDEX；
- 修改对象结构：ALTER TABLE、ALTER VIEW；
- 删除对象：DROP TABLE、DROP VIEW、DROP INDEX；
- 定义完整性约束：主键、外键、唯一、非空、检查约束；
- 定义模式、表空间、用户、角色等数据库结构对象。

解析：

DDL 的关键词是“定义结构”，不是操作具体数据。

如果题目问 DML，则应回答 SELECT、INSERT、UPDATE、DELETE 等数据操作。

2. 原题：关系数据库设计中，至少应满足的规范化条件是什么？

参考答案：

一般认为关系数据库设计至少应达到第三范式 3NF。

如果只达到 1NF，只能保证属性原子；如果达到 2NF，消除了非主属性对候选码的部分依赖；达到 3NF 后，还能消除非主属性对候选码的传递依赖。

解析：

考试中常见表述是：“工程上至少满足 3NF，在性能要求较高时可以适度反规范化。”

如果题目强调更严格理论设计，也可以进一步追求 BCNF，但 BCNF 分解可能不能保持函数依赖。

3. 原题：判断分解后的关系模式是否合理的两个重要标志是什么？

参考答案：

两个重要标志是：

1. 无损连接性；

2. 保持函数依赖。

解析：

无损连接保证分解后的表重新连接时不产生伪元组、不丢失原关系信息；保持函数依赖保证原来的业务约束可以在分解后的关系中检查。

代表性方法：

- 判断无损连接：看公共属性是否能函数决定某一分解子关系；
- 判断保持依赖：看原函数依赖是否能由分解后各关系投影出的依赖推出。

4. 原题：基于多表的视图，可以完成哪些操作？不能完成哪些操作？

参考答案：

基于多表的视图通常可以完成查询操作，也可以用于隐藏复杂连接、限制用户访问字段、简化应用 SQL。

但多表视图通常不能直接完成复杂的 INSERT、UPDATE、DELETE 操作，尤其包含以下情况时一般不可更新：

- 包含连接；
- 包含聚合函数；
- 包含 GROUP BY；
- 包含 DISTINCT；
- 包含集合运算；
- 视图列不是基础表列的直接映射。

解析：

视图是虚表，本质上保存的是查询定义。

简单单表视图可能可更新，多表复杂视图通常只适合作为查询接口。

5. 原题：实体之间的联系有哪几种？分别举例说明？

参考答案：

实体之间常见联系有三种：

- 一对一 1:1：一个人对应一个身份证号，一个身份证号也只对应一个人；
- 一对多 1:n：一个学院有多个学生，一个学生属于一个学院；
- 多对多 m:n：一个学生可以选多门课程，一门课程也可以被多个学生选择。

解析：

E-R 模型转换为关系模型时：

- 1:1 可在任一方加入外键；
- 1:n 通常在 n 的一方加入外键；

- m:n 必须转换成一个独立联系表。

6. 原题：简述等值连接与自然连接的异同。

参考答案：

相同点：

- 都属于连接运算；
- 都可以根据两个关系中属性值相等进行元组组合。

不同点：

- 等值连接需要显式给出连接条件，如 $R.A = S.B$ ；
- 自然连接自动按同名属性做等值连接；
- 等值连接结果中通常保留两个参与比较的属性列；
- 自然连接会去掉重复的同名属性列。

解析：

自然连接更简洁，但如果两个表有同名但语义不同的字段，可能产生错误连接。

考试中写关系代数时，自然连接常写作 $R \bowtie S$ ，等值连接常写作 $R \bowtie_{\{R.A=S.B\}} S$ 。

7. 原题：简述 where 子句与 having 子句的区别。

参考答案：

WHERE 用于分组前筛选行，HAVING 用于分组后筛选组。

示例：

```
SELECT dept_id, COUNT(*) AS cnt
FROM student
WHERE gender = '男'
GROUP BY dept_id
HAVING COUNT(*) > 10;
```

解析：

上例中，WHERE 先筛选男生，再按学院分组；HAVING 再筛选人数超过 10 的学院。聚合函数通常不能写在 WHERE 条件中，而应写在 HAVING 中。

8. 原题：简述两阶段锁协议的具体含义。

参考答案：

两阶段锁协议 2PL 要求事务的加锁和解锁分为两个阶段：

1. 增长阶段：事务可以申请锁，但不能释放锁；

2. 收缩阶段：事务可以释放锁，但不能再申请新锁。

解析：

满足两阶段锁协议的调度一定是冲突可串行化调度。

严格两阶段锁要求事务持有的排他锁直到提交或回滚才释放，可以避免级联回滚。

9. 原题：简述数据库中为何要进行并发控制？

参考答案：

数据库进行并发控制，是为了在多个事务同时执行时保证数据一致性和隔离性。

如果没有并发控制，可能出现：

- 丢失修改；
- 脏读；
- 不可重复读；
- 幻读；
- 错误汇总。

解析：

并发控制的目标不是禁止并发，而是在保证正确性的前提下提高系统吞吐量。

常见技术包括锁、时间戳、多版本并发控制 MVCC 等。

10. 原题：举例说明数据库中死锁含义及其解决方法？

参考答案：

死锁是指两个或多个事务互相等待对方释放资源，导致都无法继续执行。

例子：

- 事务 T1 已锁定数据 A，想申请数据 B；
- 事务 T2 已锁定数据 B，想申请数据 A；
- 两者互相等待，形成死锁。

解决方法：

- 死锁预防：统一加锁顺序、一次性申请所有锁；
- 死锁检测：构造等待图，发现环则说明死锁；
- 死锁解除：回滚其中一个事务，释放其持有的锁；
- 超时机制：等待超过阈值自动回滚。

第二部分：分析题

1. 原题：吉林大学的 BBS 系统，用户可以以游客身份浏览留言帖，但如果想要发言或者回复留言，则必须先登录方可留言，为什么？在数据库端如何实

现?

参考答案:

原因是发言和回复属于会改变数据库状态的操作, 必须知道操作者身份, 才能保证数据安全、权限控制和责任追踪。游客只能读, 登录用户才能写。

数据库端实现方式:

- 建立用户表 `Users(User_ID, User_Name, Password, Role)` ;
- 留言表保存发言人外键, 如 `Posts(Post_ID, User_ID, Content, Post_Time)` ;
- 对游客只授予查询权限;
- 对登录用户授予插入留言和回复的权限;
- 在应用层或数据库层检查 `User_ID` 是否存在;
- 用外键保证留言必须对应合法用户。

示例:

```
CREATE TABLE Posts (  
    Post_ID    CHAR(20) PRIMARY KEY,  
    User_ID    CHAR(20) NOT NULL,  
    Content    VARCHAR2(1000) NOT NULL,  
    Post_Time  DATE DEFAULT SYSDATE,  
    CONSTRAINT FK_Post_User  
        FOREIGN KEY (User_ID) REFERENCES Users(User_ID)  
);
```

解析:

代表性方法是从“读写权限不同”入手: 浏览是 `SELECT`, 发言是 `INSERT`, 回复可能是 `INSERT` 或 `UPDATE`。

数据库端可以通过用户、角色、外键、触发器、审计日志共同实现。

2. 原题: 从银行卡向学校一卡通转账时, 如果金额已从银行卡扣除, 但一卡通还未到账, 机器死机无法正常工作, 数据库处于什么状态? 如何解决?

参考答案:

数据库处于不一致状态。

转账应该是一个事务, 扣银行卡和加一卡通必须同时成功或同时失败。现在只完成了扣款, 没有完成入账, 违反事务的原子性和一致性。

解决方法:

- 将扣款和入账放在同一个事务中;
- 若全部成功, 则 `COMMIT` ;
- 若中途故障, 则利用日志回滚已完成的扣款操作;

- 系统恢复时根据日志进行撤销 UNDO 或重做 REDO。

示例：

```
BEGIN
    UPDATE BankCard
        SET Balance = Balance - 100
        WHERE Card_ID = 'B001'
        AND Balance >= 100;

    UPDATE CampusCard
        SET Balance = Balance + 100
        WHERE Card_ID = 'C001';

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
END;
/
```

解析：

这类题答题关键词是事务、原子性、日志、恢复、回滚。
不要只回答“重新转账”，要说明数据库通过事务机制解决。

3. 原题：支付宝账号通常与某一银行卡绑定，该卡不可透支。若当前卡内余额 100 元，另一人使用绑定银行卡刷卡消费 100 元，能否支付成功？若不可以，在数据库端如何控制？

参考答案：

如果余额正好为 100 元，且没有其他并发扣款，理论上可以支付成功；但如果支付宝和刷卡消费并发发生，必须在数据库端控制，防止两个事务同时读到余额 100 元并都扣款成功。

数据库端控制方法：

- 在扣款前检查余额；
- 对账户记录加排他锁；
- 在一个事务中完成“检查余额 + 扣款”；
- 设置检查约束保证余额不能小于 0。

示例：

```
UPDATE Account
    SET Balance = Balance - 100
```

```
WHERE Account_ID = 'A001'
AND Balance >= 100;
```

若该语句影响行数为 1，则扣款成功；若为 0，则余额不足。
也可以使用：

```
SELECT Balance
FROM Account
WHERE Account_ID = 'A001'
FOR UPDATE;
```

解析：

代表性方法是使用“条件更新”或“行级锁”。
不要先 SELECT 再无锁 UPDATE，否则并发场景下会产生超额扣款。

4. 原题：学校所有学生的数据都存在同一个表内，但各学院教务管理员登录后只能看到自己学院的学生，在数据库端如何实现？

参考答案：

可通过视图和权限控制实现。
为每个学院建立只包含本学院学生的视图，再把视图查询权限授予对应学院管理员。

示例：

```
CREATE OR REPLACE VIEW V_STUDENT_CS AS
SELECT *
FROM Student
WHERE College_ID = 'CS';

GRANT SELECT ON V_STUDENT_CS TO cs_admin;
```

也可以用行级安全策略实现，根据当前登录用户自动过滤学院。

解析：

这题考“同表不同用户只能看部分行”，关键词是视图、授权、行级访问控制。
如果只在前端过滤，安全性不够；数据库端也应限制权限。

第三部分：设计题

原题

吉林大学为了提高实验教学质量，为每门实验课配备若干指导教师，指导学生实验。学院开设多门实验课程供学生选择，每个学生可以选择一到多门实验课，每门课程有若干指导教师，每

个教师可以参与多门实验课指导；在实验课中，一位教师负责指导多个学生，而一个学生有且只有一名指导教师。

要求：

1. 根据需求画出 E-R 模型；
2. 将 E-R 模型转化为关系模式，并说明完整性约束。实体属性自行设定，每个实体不少于三个属性，不多于五个属性。

参考答案：

可以抽象出实体：

- 学生：Student(Sid, Sname, Sex, Major)；
- 实验课：LabCourse(Cid, Cname, Credit, Term)；
- 教师：Teacher(Tid, Tname, Title, Phone)；
- 学院：College(College_ID, College_Name, Office)。

主要联系：

- 学院开设实验课：College 与 LabCourse 为 1:n；
- 学生选实验课：Student 与 LabCourse 为 m:n；
- 教师指导实验课：Teacher 与 LabCourse 为 m:n；
- 在某门实验课中，教师指导学生：可建联系表记录学生、课程、教师。

关系模式：

```
College(College_ID, College_Name, Office)
Student(Sid, Sname, Sex, Major, College_ID)
LabCourse(Cid, Cname, Credit, Term, College_ID)
Teacher(Tid, Tname, Title, Phone)
TeacherCourse(Tid, Cid)
StudentCourse(Sid, Cid, Tid)
```

完整性约束：

- College_ID、Sid、Cid、Tid 分别为主键；
- Student.College_ID 引用 College；
- LabCourse.College_ID 引用 College；
- TeacherCourse(Tid, Cid) 为联合主键，并分别引用教师和课程；
- StudentCourse(Sid, Cid) 为联合主键，表示学生选课；
- StudentCourse.Tid 引用 Teacher；
- 还应保证 (Tid, Cid) 存在于 TeacherCourse 中，即指导该学生的教师必须确实参与该课程指导。

解析：

代表性方法：

1. 先找实体：学生、课程、教师、学院；
2. 再判断联系基数：学生选课是多对多，教师授课也是多对多；
3. 最后处理“某课中一个学生只有一个指导教师”这个业务约束，应落在 StudentCourse 表的 Tid 字段上。

第四部分：应用题

1. 原题：仓库-物料存储系统

关系模式如下：

```
W(Wid, WN, Addr)      -- 仓库号, 仓库名称, 地址
M(Mid, MN, Price, P_Time, Q_Time) -- 物料号, 物料名称, 价格, 生产日期, 保质期
ST(Wid, Mid, SNum)    -- 仓库号, 物料号, 存储数量
```

1. 用关系代数表达式，查询存储了全部物料的仓库名称

参考答案：

```

$$\pi_{WN}(\sigma_{W \bowtie (\pi_{Wid, Mid}(ST) \div \pi_{Mid}(M))}$$

```

解析：

“存储了全部物料”是典型除法题。

先用 ST 得到仓库与物料的对应关系，再除以全部物料编号集合，得到满足“对每个物料都存储”的仓库号。

2. 用关系代数表达式，查询所有存储数量高于 100 的物料号

参考答案：

```

$$\pi_{Mid}(\sigma_{SNum > 100}(ST))$$

```

解析：

先选择 $SNum > 100$ 的库存记录，再投影物料号。

3. 用 SQL 查询所有没有生产日期的物料号和物料名称

参考答案：

```
SELECT Mid, MN
FROM M
WHERE P_Time IS NULL;
```

解析：

空值不能写成 `P_Time = NULL`，必须使用 `IS NULL`。

4. 用 SQL 建立视图 v，统计截至今天还有三个月就要到保质期的物料号和物料名称

参考答案：

```
CREATE OR REPLACE VIEW V AS
SELECT Mid, MN
FROM M
WHERE ADD_MONTHS(P_Time, Q_Time) <= ADD_MONTHS(SYSDATE, 3)
AND ADD_MONTHS(P_Time, Q_Time) >= SYSDATE;
```

解析：

这里把 `Q_Time` 理解为保质期月数。

如果 `Q_Time` 是日期型“到期日期”，则可写：

```
CREATE OR REPLACE VIEW V AS
SELECT Mid, MN
FROM M
WHERE Q_Time BETWEEN SYSDATE AND ADD_MONTHS(SYSDATE, 3);
```

考试时要根据题目字段含义说明自己的理解。

5. 用 SQL 修改物料名为“海绵”的库存，增加 100 件

参考答案：

```
UPDATE ST
SET SNum = SNum + 100
WHERE Mid IN (
    SELECT Mid
    FROM M
    WHERE MN = '海绵'
);
```

解析：

库存数量在 `ST` 表中，物料名称在 `M` 表中，因此需要子查询先找到物料号。

如果一个名称对应多个物料号，`IN` 比 `=` 更稳妥。

2. 原题：设关系模式 $R(A, B, C, D, E, F, G)$ ，函数依赖集 $F=\{AB\rightarrow C, B\rightarrow DE, C\rightarrow G, G\rightarrow A\}$ ，求模式 R 的所有候选码。

参考答案：

候选码为：

ABF, BCF, BFG

解题过程：

右部出现的属性有 A, C, D, E, G ，没有出现在任何右部的属性是 B, F 。
因此每个候选码必须包含 B, F 。

计算闭包：

$(ABF)^+$ ：
 $AB\rightarrow C, C\rightarrow G, G\rightarrow A, B\rightarrow DE$
得到 A, B, C, D, E, F, G

$(BCF)^+$ ：
 $C\rightarrow G, G\rightarrow A, AB\rightarrow C, B\rightarrow DE$
得到 A, B, C, D, E, F, G

$(BFG)^+$ ：
 $G\rightarrow A, AB\rightarrow C, B\rightarrow DE, C\rightarrow G$
得到 A, B, C, D, E, F, G

BF 本身不能推出 $A/C/G$ ，所以还需补充 A 、 C 或 G 中的一个。
故候选码是 ABF 、 BCF 、 BFG 。

方法总结：

1. 先找没有出现在任何依赖右部的属性，它们必在候选码中；
2. 再逐步补属性求闭包；
3. 能推出全部属性且没有多余属性的集合就是候选码。

3. 原题：设关系 $R(A, B, C, D, E)$ ，函数依赖 $F=\{ABC\rightarrow DE, BC\rightarrow D, D\rightarrow E\}$ ，将关系 R 逐步分解为 3NF。

参考答案：

候选码为 ABC 。

因为 $ABC\rightarrow DE$ ，所以 $ABC^+=\{A, B, C, D, E\}$ 。

存在问题：

- $BC \rightarrow D$ 是非主属性 D 对候选码 ABC 的部分依赖；
- $D \rightarrow E$ 使 E 对候选码存在传递依赖。

可分解为：

```
R1(B, C, D)
R2(D, E)
R3(A, B, C)
```

解析：

- $R1(B, C, D)$ 保持 $BC \rightarrow D$ ；
- $R2(D, E)$ 保持 $D \rightarrow E$ ；
- $R3(A, B, C)$ 保存原候选码，保证无损连接；
- 通过 $BC \rightarrow D$ 和 $D \rightarrow E$ 可以推出 $ABC \rightarrow DE$ ，因此依赖可保持。

4. 原题：设如图所示调度 S ，判断 S 是否为冲突可串行化调度；如果是，给出与 S 等价的一个串行调度。

参考答案：

根据图中操作可读出主要顺序：

```
T1: R(A), W(A), W(B)
T2: R(A), W(A), R(B)
T3: R(B), W(B), R(C), R(D)
T4: R(D), R(C), W(C)
```

构造冲突边：

- 对 A ： $T1$ 的 $W(A)$ 在 $T2$ 的 $R(A)$, $W(A)$ 之前，所以有 $T1 \rightarrow T2$ ；
- 对 B ： $T3$ 的 $R(B)$, $W(B)$ 在 $T1$ 的 $W(B)$ 和 $T2$ 的 $R(B)$ 之前，所以有 $T3 \rightarrow T1$ 、 $T3 \rightarrow T2$ ；
- 对 C ： $T3$ 的 $R(C)$ 在 $T4$ 的 $W(C)$ 之前，所以有 $T3 \rightarrow T4$ 。

优先图无环，因此该调度是冲突可串行化调度。

一个等价串行调度为：

```
T3 → T1 → T2 → T4
```

解析：

代表性方法：

1. 只看不同事务对同一数据项的冲突操作；

2. 冲突操作至少有一个是写；
 3. 按调度先后画边；
 4. 优先图无环，则冲突可串行化。
-

二、2016-2017 学年第 2 学期 2015 级《数据库原理》考试试题 (A) 解析

第一部分：简答题

1. 原题：数据库中常用的完整性约束包括哪些？

参考答案：

常见完整性约束包括：

- 实体完整性：主键不能为空且唯一；
- 参照完整性：外键必须引用已存在的主键或候选键；
- 用户自定义完整性：根据业务定义的非空、唯一、检查约束等；
- 域完整性：字段取值必须属于规定类型和范围。

2. 原题：简述数据库系统与文件系统的主要区别。

参考答案：

数据库系统和文件系统的区别主要体现在：

- 数据库系统数据结构化程度高，文件系统数据结构化弱；
- 数据库支持共享和并发控制，文件系统共享控制能力弱；
- 数据库能减少冗余并维护一致性；
- 数据库支持事务、恢复、安全和完整性约束；
- 文件系统通常由应用程序自行解释和维护数据。

3. 原题：一般情况下，关系 R 与关系 S 要进行自然连接，需要满足什么条件？

参考答案：

自然连接要求两个关系具有同名属性，并且这些同名属性语义相同、类型可比。连接时系统会自动按同名属性取值相等进行匹配，并在结果中只保留一份同名属性。

4. 原题：为什么要在数据库中引入事务的概念？

参考答案：

引入事务是为了把多个数据库操作组织成一个逻辑工作单位，保证这些操作要么全部成功，要么全部失败。

事务用于保证 ACID 特性，即原子性、一致性、隔离性和持久性。

5. 原题：已知视图 `faculty` 的定义： `create view faculty as select id, name, dept_name from instructor`；当发布 `insert into faculty values('30765', 'Green', 'Music')` 命令时，数据库系统如何执行？

参考答案：

该视图是基于单表 `instructor` 的简单投影视图，没有聚合、分组、连接和表达式列，因此通常可以更新。

数据库会把对视图的插入转换为对基础表 `instructor` 的插入。

等价操作近似为：

```
INSERT INTO instructor(id, name, dept_name)
VALUES ('30765', 'Green', 'Music');
```

如果 `instructor` 表中还有其他非空且无默认值字段，则插入会失败，因为视图没有提供那些字段的值。

6. 原题：用基本关系代数表达式来表示 $R \cap S$ 。

参考答案：

$$R \cap S = R - (R - S)$$

解析：

交集可由差运算表示：先从 R 中去掉属于 S 的元组，剩下 $R-S$ ；再用 R 减去这部分，就得到同时属于 R 和 S 的元组。

7. 原题：银行按储蓄金额执行不同利率。低于 10 万元年利率 4.2%，10 万元以上（包括 10 万）年利率 4.5%。账户表

`account(aid, balance, branch_name)`，写 SQL 并说明如何保证结果正确性。

参考答案：

```
SELECT aid,
       balance,
       CASE
         WHEN balance >= 100000 THEN balance * 0.045
         ELSE balance * 0.042
       END AS yearly_interest
FROM account;
```

为了保证结果正确性：

- 使用 CASE WHEN 明确区分金额区间；
- 边界值 100000 应放入高利率区间；
- 对余额设置非负检查约束；
- 如果计算并入账，应放在事务中执行。

8. 原题：什么样的关系模式满足 BCNF？

参考答案：

关系模式 R 满足 BCNF，当且仅当对于 R 上每一个非平凡函数依赖 $X \rightarrow Y$ ，X 都是 R 的超码。

解析：

BCNF 比 3NF 更严格。

3NF 允许右部属性是主属性的某些例外，而 BCNF 不允许，要求每个决定因素都必须是超码。

9. 原题：关系数据库中，超码、候选码、主码有什么区别？

参考答案：

- 超码：能唯一标识元组的属性集，可以含多余属性；
- 候选码：最小超码，去掉任何属性都不能唯一标识元组；
- 主码：从候选码中选定的一个，用作表的主要标识。

例子：

若 Student(Sid, IDCard, Name) 中 Sid 和 IDCard 都唯一，则二者都是候选码；选择 Sid 作为主码；{Sid, Name} 是超码但不是候选码。

10. 原题：关系代数中，选择操作的功能是什么？

参考答案：

选择操作 σ 用于从关系中筛选满足给定条件的元组，相当于 SQL 的 WHERE 行筛选。

示例：

```
 $\sigma_{\text{balance} > 10000}(\text{account})$ 
```

表示选出余额大于 10000 的账户元组。

11. 原题：简述数据库中引入封锁机制的优缺点。

参考答案：

优点：

- 保证并发事务的隔离性；
- 防止丢失修改、脏读等并发异常；
- 可实现可串行化调度。

缺点：

- 降低并发度；
- 可能产生死锁；
- 加锁和解锁有系统开销；
- 粒度过大会影响性能，粒度过小管理复杂。

12. 原题：数据库中，调度指的是什么？

参考答案：

调度是指多个事务并发执行时，各事务读写操作在时间上的交错执行顺序。

调度是否正确，通常用可串行化、冲突可串行化、可恢复性等标准判断。

13. 原题：事务由哪几个状态组成？手机支付已提交，但网络信号消失导致支付失败，此时事务处于何种状态？

参考答案：

事务常见状态包括：

- 活动状态；
- 部分提交状态；
- 提交状态；
- 失败状态；
- 中止状态。

若“已提交”只是客户端发出了提交请求，但数据库端未真正完成提交，网络故障导致支付失败，则事务处于失败状态，之后应回滚到中止状态。

若数据库已经完成提交，则应处于提交状态，网络只影响客户端收到结果。

解析：

这题关键是区分“用户看到的提交”和“数据库真正提交”。考试中通常按支付失败处理，回答失败/中止并由数据库回滚。

14. 原题：在 MySQL 中定义 foreign key 时，与标准 SQL 有何区别？

参考答案：

标准 SQL 中外键是关系数据库约束的一部分；MySQL 中外键能否真正生效与存储引擎有关。

例如 InnoDB 支持外键，而 MyISAM 不真正执行外键约束。MySQL 还要求被引用列必须有索引，且引用列和被引用列类型、长度等应兼容。

15. 原题：MySQL 不支持 INTERSECT 语句，可以用哪些语句来代替？

参考答案：

可以使用内连接或 IN / EXISTS 代替。

```
SELECT a.*
FROM R a
JOIN S b
  ON a.key_col = b.key_col;
```

或：

```
SELECT *
FROM R
WHERE key_col IN (SELECT key_col FROM S);
```

16. 原题：数据库中如何判断死锁是否发生？

参考答案：

可以建立事务等待图。

图中节点表示事务，边 $T_i \rightarrow T_j$ 表示 T_i 正在等待 T_j 持有的资源。若等待图中存在环，则说明发生死锁。

17. 原题：什么样的调度一定能够保证数据的一致性？

参考答案：

可串行化调度能够保证数据一致性。

如果每个事务单独执行都能保持数据库一致性，则任意与某个串行调度等价的可串行化调度也能保持一致性。

18. 原题：为了保证数据库的并发性，引入了哪些锁？

参考答案：

常见锁包括：

- 共享锁 S：用于读；
- 排他锁 X：用于写；
- 意向共享锁 IS；
- 意向排他锁 IX；
- 共享意向排他锁 SIX。

也可按粒度分为表锁、页锁、行锁。

19. 原题：为什么事务非正常结束会影响数据库数据的正确性？请举例说明。

参考答案：

事务非正常结束可能使一组本应整体完成的操作只执行了一部分，从而破坏数据库一致性。

例子：转账时从账户 A 扣款成功，但给账户 B 加款前系统崩溃。此时总金额减少，数据库处于不一致状态。

解决办法是使用事务日志和恢复机制，对未提交事务执行撤销，对已提交事务执行重做。

20. 原题：利用 Armstrong 公理证明伪传递率：若有 $A \rightarrow B$ ， $CB \rightarrow D$ ，则有 $AC \rightarrow D$ 。

证明：

由 $A \rightarrow B$ ，根据增广律，两边同时加上 C，得：

$$AC \rightarrow BC$$

由已知：

$$CB \rightarrow D$$

即：

$$BC \rightarrow D$$

根据传递律，由 $AC \rightarrow BC$ 和 $BC \rightarrow D$ ，得：

$$AC \rightarrow D$$

证毕。

第二部分：分析题

1. 原题：两个人同时购买同一航班最后一张经济舱票，数据库中要如何控制？

参考答案：

应使用事务和并发控制，保证同一张票不能被两个事务同时售出。

可采用行级锁或条件更新。

示例：

```
UPDATE FlightSeat
  SET remain_seats = remain_seats - 1
 WHERE flight_no = 'CZ6147'
    AND flight_date = DATE '2017-07-02'
    AND cabin_type = '经济舱'
    AND remain_seats > 0;
```

若影响行数为 1，则购票成功；若为 0，则余票不足。

解析：

代表性方法是“检查余票”和“扣减余票”合并为一条原子更新语句，或先 `SELECT ... FOR UPDATE` 锁住航班余票记录。

2. 原题：在线手机游戏支持离线操作，断网时可离线玩，联网后再同步，可能涉及数据库哪些概念？

参考答案：

可能涉及：

- 本地缓存；
- 数据同步；
- 事务；
- 冲突检测与冲突解决；
- 日志；
- 时间戳或版本号；
- 最终一致性；
- 并发控制。

解析：

离线操作不是立即写入服务器数据库，联网后需要把本地操作合并到服务器。

若同一数据在多端被修改，就需要冲突检测策略，例如以服务器为准、以时间戳为准或人工合并。

3. 原题：用户到银行开了储蓄账户，可对账户存钱、取钱等。分别对应数据库中什么操作？在 ATM 取钱余额不足时，ATM 不做支付，如何定义规则？

参考答案：

数据库操作：

- 开户： `INSERT` ；
- 存钱： `UPDATE balance = balance + 金额` ；
- 取钱： `UPDATE balance = balance - 金额` ；

- 销户：DELETE 或更新状态；
- 查询余额：SELECT 。

余额不足不支付可用条件更新实现：

```
UPDATE account
  SET balance = balance - :money
 WHERE aid = :aid
  AND balance >= :money;
```

若更新行数为 0，说明余额不足或账户不存在。

也可增加约束：

```
ALTER TABLE account
ADD CONSTRAINT CK_ACCOUNT_BALANCE CHECK (balance >= 0);
```

4. 原题：一卡通与银行绑定，当一卡通余额小于设定金额时，可通过银行卡自动转账，如何实现自动转账？转账金额在数据库端如何设定？

参考答案：

可以在一卡通消费或余额更新后，通过触发器或存储过程检查余额。如果余额低于阈值，则从绑定银行卡扣款并给一卡通充值。

示例：

```
CREATE TABLE AutoPayRule (
  Card_ID          CHAR(20) PRIMARY KEY,
  Bank_Account     CHAR(20) NOT NULL,
  Min_Balance      NUMBER(10,2) NOT NULL,
  Transfer_Amount  NUMBER(10,2) NOT NULL
);
```

触发器思路：

```
CREATE OR REPLACE TRIGGER TRG_AUTOPAY
AFTER UPDATE OF Balance ON CampusCard
FOR EACH ROW
WHEN (NEW.Balance < OLD.Balance)
DECLARE
  v_min NUMBER;
  v_amt NUMBER;
  v_bank CHAR(20);
BEGIN
  SELECT Min_Balance, Transfer_Amount, Bank_Account
```

```

        INTO v_min, v_amt, v_bank
        FROM AutoPayRule
        WHERE Card_ID = :NEW.Card_ID;

    IF :NEW.Balance < v_min THEN
        UPDATE BankAccount
            SET Balance = Balance - v_amt
            WHERE Account_ID = v_bank
            AND Balance >= v_amt;

        UPDATE CampusCard
            SET Balance = Balance + v_amt
            WHERE Card_ID = :NEW.Card_ID;
    END IF;
END;
/

```

解析：

“设定金额”最好不要写死在程序中，应保存到规则表中。

自动转账必须放在事务中，避免银行卡扣款成功但一卡通充值失败。

第三部分：设计题

原题

某金融公司为客户提供理财服务。公司提供股票、基金、外币等多种投资理财项目，并且为每位客户配备一个理财经理，为客户提供投资咨询服务。客户可以购买一种或多种理财产品，根据买卖时间和购买金额计算收益。每个客户还可以推荐下线客户，每个客户只能有一个上线。

要求：

1. 根据需求画 E-R 模型；
2. 转换为关系模型，并说明完整性约束。

参考答案：

实体：

```

Customer(Cid, Cname, Phone, Level, Parent_Cid)
Manager(Mid, Mname, Phone, Dept)
Product(Pid, Pname, Ptype, Risk_Level, Price)
Purchase(Cid, Pid, Buy_Time, Amount, Income)

```

联系：

- 理财经理服务客户：Manager 1:n Customer；
- 客户购买产品：Customer m:n Product，联系属性有购买时间、金额、收益；

- 客户推荐客户：客户实体自联系，Parent_Cid 表示上线客户。

关系模式：

```
Manager(Mid, Mname, Phone, Dept)
Customer(Cid, Cname, Phone, Level, Mid, Parent_Cid)
Product(Pid, Pname, Ptype, Risk_Level, Price)
Purchase(Cid, Pid, Buy_Time, Amount, Income)
```

完整性约束：

- Manager.Mid、Customer.Cid、Product.Pid 为主键；
- Customer.Mid 外键引用 Manager(Mid)；
- Customer.Parent_Cid 外键引用 Customer(Cid)，可为空；
- Purchase(Cid, Pid, Buy_Time) 可作为联合主键；
- Purchase.Cid 引用 Customer；
- Purchase.Pid 引用 Product；
- Amount ≥ 0 ，Income 可正可负。

解析：

代表性方法：

看到“客户可以购买多种产品，产品也可被多个客户购买”，应转成购买联系表；看到“每个客户只有一个上线”，可用客户表中的自引用外键表示。

第四部分：计算题

1. 原题：判断图中调度 S 是否为冲突可串行化调度。

参考答案：

根据图中操作，冲突边中至少存在：

- $T1 \rightarrow T2$ ：T1 先读/写 A，T2 后写 A；
- $T2 \rightarrow T1$ ：T2 先写 A，T1 后写 A；
- $T3 \rightarrow T1$ 、 $T1 \rightarrow T3$ 等关于 A 的冲突。

优先图存在环，因此该调度不是冲突可串行化调度。

解析：

判断调度题不需要关心所有非冲突操作，只要找到一个环即可判定不是冲突可串行化。本题中 A 上的交错写操作已足以构造环。

2. 原题：交通肇事关系模式 $R(A, B, C, D, E, F, G, H, I)$ ，语义为：A 交通肇事编号，B 发生时间，C 具体原因，D 扣分，E 罚款金额，F 车牌号，G 车型

及颜色，H 驾照号，I 驾照已扣分。司机与车辆 1:1，车辆与违章 m:n；车牌号唯一，驾照号唯一。要求给出函数依赖并分解为 3NF。

参考答案：

可根据语义给出主要函数依赖：

```
A → B C F H
C → D E
F → G H
H → F I
```

含义：

- 肇事编号确定发生时间、原因、车辆和驾驶员；
- 违章原因确定扣分和罚款金额；
- 车牌号唯一，可确定车型颜色和驾驶员；
- 驾照号唯一，可确定车辆和已扣分。

3NF 分解：

```
Violation(A, B, C, F, H)
ViolationRule(C, D, E)
Vehicle(F, G, H)
Driver(H, I)
```

主键：

- Violation 主键为 A；
- ViolationRule 主键为 C；
- Vehicle 主键为 F；
- Driver 主键为 H。

解析：

题目语义有一定简化，关键是把“事件事实”“违章规则”“车辆信息”“驾驶员信息”拆开。这样可以减少车型、罚款规则和驾驶员扣分信息的重复存储。

第五部分：应用题

原题

某连锁超市商品管理系统：

```
MD(Mid, Mname, MRnum, Maddress) -- 门店
SP(Pid, Pname, Price)           -- 商品
```

MP(Mid, Pid, Pnum)

-- 门店商品数量

1. 用关系代数查询店员人数不少于 50 人的门店名称和地址

参考答案：

```
 $\pi_{Mname, Maddress}(\sigma_{MRnum \geq 50}(MD))$ 
```

2. 用关系代数查询：找出至少供应了代码为“256”的商店所供应的全部商品的其他商店的商店名和地址

参考答案：

```
 $\pi_{Mname, Maddress}(\text{MD} \bowtie (\text{MD} \div \pi_{Pid}(\sigma_{Mid='256'}(MP))) - \pi_{Mid}(\sigma_{Mid='256'}(MD)))$ 
```

解析：

这是典型除法题：先取 256 号门店供应的商品集合，再找供应了该集合全部商品的门店。

3. 用 SQL 查询库存数量小于 10 件的商品名称及所在门店名称

参考答案：

```
SELECT sp.Pname, md.Mname
FROM MP mp
JOIN SP sp ON mp.Pid = sp.Pid
JOIN MD md ON mp.Mid = md.Mid
WHERE mp.Pnum < 10;
```

4. 用 SQL 查询各门店名称和所拥有商品的总价格

参考答案：

```
SELECT md.Mname,
       SUM(sp.Price * mp.Pnum) AS total_value
FROM MD md
JOIN MP mp ON md.Mid = mp.Mid
JOIN SP sp ON mp.Pid = sp.Pid
GROUP BY md.Mname;
```

5. 将“256”号门店的“泉阳泉”数量增加 2000

参考答案：

```
UPDATE MP
  SET Pnum = Pnum + 2000
 WHERE Mid = '256'
  AND Pid IN (
    SELECT Pid
    FROM SP
    WHERE Pname = '泉阳泉'
  );
```

三、2018 年 6 月 25 日《数据库》考试试题解析

第一部分：简答题

1. 原题：为什么要为数据库加严格两阶段锁或强两阶段锁？

参考答案：

严格两阶段锁和强两阶段锁用于保证并发调度的正确性，避免脏读、级联回滚和不可恢复调度。

严格两阶段锁要求事务提交或回滚前不释放排他锁；强两阶段锁通常要求事务提交前不释放所有锁。

解析：

普通 2PL 保证冲突可串行化，但不一定避免级联回滚。严格 2PL 更适合实际数据库系统。

2. 原题：为什么要在数据库中引入事务的概念？

参考答案：

事务把多个数据库操作封装为一个逻辑单元，保证操作整体满足 ACID 特性。它用于解决转账、支付、订票等“要么全部成功，要么全部失败”的业务。

3. 原题：为什么要在数据库中设置多种不同粒度的锁？

参考答案：

不同粒度锁用于在并发度和管理开销之间取得平衡。

- 粒度大：如表锁，管理简单但并发度低；
- 粒度小：如行锁，并发度高但锁管理开销大；
- 多粒度锁结合意向锁，可以让系统同时支持表级和行级加锁。

4. 原题：为什么要对数据库的调度进行可串行化判别，其实际意义是什么？

参考答案：

可串行化判别用于判断并发调度是否等价于某个串行调度。

如果一个调度可串行化，则在每个事务本身正确的前提下，并发执行结果也能保持数据库一致性。

实际意义是：允许事务交错执行提高效率，同时保证结果像串行执行一样正确。

5. 原题：若对数据库进行 BCNF 范式分解，导致没有保持依赖，在数据库设计中如何解决？

参考答案：

可采用以下方法：

- 改用 3NF 分解，因为 3NF 分解通常可以同时保证无损连接和保持依赖；
- 在应用程序、触发器或断言中额外检查未保持的依赖；
- 根据业务重要性在 BCNF 与依赖保持之间折中；
- 必要时保留一定冗余。

解析：

BCNF 更严格，但不保证保持依赖。考试中常答：“若依赖保持更重要，可采用 3NF。”

6. 原题：数据库中，实体的完整性是如何被保证的？

参考答案：

实体完整性通过主键约束保证。

主键值必须唯一且不能为空，从而每个元组都能被唯一识别。

示例：

```
CREATE TABLE Student (  
    Sid CHAR(10) PRIMARY KEY,  
    Sname VARCHAR2(40) NOT NULL  
);
```

7. 原题：如何降低数据库中数据的冗余度？

参考答案：

主要方法包括：

- 进行规范化设计；
- 分解存在部分依赖和传递依赖的关系；
- 使用主键、外键组织表间关系；
- 将多值属性和多对多联系拆成独立表；

- 避免重复存储可计算字段。

8. 原题：如何标识一个弱实体集？

参考答案：

弱实体不能仅靠自身属性唯一标识，必须依赖强实体的主键和自身的部分键共同标识。
例如订单明细依赖订单存在，可用 (Order_ID, Item_No) 作为主键。

E-R 图中弱实体通常用双矩形表示，标识联系用双菱形表示。

9. 原题：在 SQL 语句中，如何表示除法运算？

参考答案：

SQL 没有直接的除法运算符，常用双重 NOT EXISTS、GROUP BY ... HAVING COUNT 或集合差来表达“对于所有”。

示例：查询选修了所有课程的学生：

```
SELECT s.Sid
FROM Student s
WHERE NOT EXISTS (
    SELECT *
    FROM Course c
    WHERE NOT EXISTS (
        SELECT *
        FROM SC
        WHERE SC.Sid = s.Sid
            AND SC.Cid = c.Cid
    )
);
```

10. 原题：关系代数中，与等值连接相比，自然连接的缺点是什么？

参考答案：

自然连接自动按同名属性连接，如果两个关系中存在同名但语义不同的属性，可能错误连接；
自然连接的连接条件不如等值连接显式，表达不够清楚。
等值连接可以明确写出连接条件，因此更可控。

第二部分：分析题

1. 原题：证明结论：如果关系 $r(R)$ 和 $s(S)$ 不含任何相同属性，即 $R \cap S = \emptyset$ ，那么 $r \bowtie s = r \times s$ 。

证明：

自然连接是对同名属性做等值匹配，并去掉重复同名属性。
当 $R \cap S = \emptyset$ 时，两个关系没有共同属性，自然连接没有任何等值约束需要判断。
因此 r 中任一元组都可以与 s 中任一元组组合，结果正是笛卡尔积。

所以：

$$r \bowtie s = r \times s$$

2. 原题：移动支付中输入正确密码并提交后发生断网，导致支付失败，分析该事务经历了哪几个状态？

参考答案：

该事务大致经历：

活动状态 → 失败状态 → 中止状态

如果数据库已经完成部分操作但尚未提交，断网或故障使事务失败，系统应回滚事务。
若数据库已经提交成功，只是客户端没有收到结果，则数据库端是提交状态，这与题目“支付失败”不同。

3. 原题：教务系统为每个学院提供固定上课教室，不同学院教务负责人登录后只能分配本学院教室资源，需要定义哪种数据库对象实现？为什么？

参考答案：

可以定义视图，并对不同学院管理员授予不同视图的权限。

示例：

```
CREATE VIEW V_ROOM_CS AS
SELECT *
FROM Classroom
WHERE College_ID = 'CS';

GRANT SELECT, UPDATE ON V_ROOM_CS TO cs_admin;
```

解析：

视图可以隐藏基础表中不属于该学院的记录，实现行级隔离。
同时配合授权机制，能让管理员只能看到和操作本学院资源。

4. 原题：学生表（学号，姓名，出生日期，学院编号）和选课信息表（学号，课程编号，成绩）。若规定学生退学时自动删除该生所有选课记录，如何定义业务逻辑？

参考答案：

可通过外键级联删除实现：

```
ALTER TABLE 选课信息表
ADD CONSTRAINT FK_SC_STUDENT
FOREIGN KEY (学号)
REFERENCES 学生表(学号)
ON DELETE CASCADE;
```

也可使用触发器：

```
CREATE OR REPLACE TRIGGER TRG_DELETE_SC
BEFORE DELETE ON 学生表
FOR EACH ROW
BEGIN
    DELETE FROM 选课信息表
    WHERE 学号 = :OLD.学号;
END;
/
```

解析：

代表性方法是优先使用数据库外键的 ON DELETE CASCADE，比触发器更简洁、更符合参照完整性语义。

第三部分：设计题

原题

某连锁超市有若干门店、若干员工和若干商品。每个门店有若干员工和一个店长，店长负责管理本门店所有员工，每个员工只在特定门店工作。各门店销售的商品基本相同，由于销售差异，每个门店拥有的特定商品数量不同。

要求：

1. 画 E-R 模型；
2. 转换为关系模式，并说明完整性约束。

参考答案：

实体：

```
Store(Store_ID, Store_Name, Address, Phone)
Employee(Emp_ID, Emp_Name, Sex, Phone)
Product(Product_ID, Product_Name, Price, Type)
```

联系：

- 门店拥有员工：Store 1:n Employee；
- 门店有一个店长：店长也是员工，可在 Store 中保存 Manager_ID；
- 门店销售商品：Store m:n Product，联系属性为库存数量。

关系模式：

```
Store(Store_ID, Store_Name, Address, Phone, Manager_ID)
Employee(Emp_ID, Emp_Name, Sex, Phone, Store_ID)
Product(Product_ID, Product_Name, Price, Type)
StoreProduct(Store_ID, Product_ID, Quantity)
```

完整性约束：

- Store_ID、Emp_ID、Product_ID 为主键；
- Employee.Store_ID 外键引用 Store；
- Store.Manager_ID 外键引用 Employee；
- StoreProduct(Store_ID, Product_ID) 为联合主键；
- Quantity >= 0；
- 业务上应保证店长属于本门店员工。

第四部分：计算题

1. 原题：判断给定调度 S 是否为冲突可串行化调度。

参考答案：

从图中读出主要操作：

```
T1: R(A), W(B)
T2: W(B), R(A), R(C)
T3: R(A), W(C)
T4: W(C), R(B)
T5: R(B)
```

冲突边中有：

- T2→T4：T2 写 B，T4 后读 B；
- T4→T2：T4 写 C，T2 后读 C。

优先图存在环：

```
T2 → T4 → T2
```

因此该调度不是冲突可串行化调度。

2. 原题：关系模式 $R(A, B, C, D, E, G, H)$ ，**函数依赖集** $F=\{A \rightarrow C, C \rightarrow A, CH \rightarrow D, C \rightarrow EG, EH \rightarrow C, CG \rightarrow DH, CE \rightarrow AG, ACD \rightarrow H\}$ 。**(1) 令** $X=BC$ ，**求** X **关于** F **的闭包；(2) 将** R **逐步分解为 3NF。**

(1) 求闭包

参考答案：

$(BC)^+ = \{A, B, C, D, E, G, H\}$

过程：

初始：BC
 $C \rightarrow A$ ，得到 A
 $C \rightarrow EG$ ，得到 E, G
 $CG \rightarrow DH$ ，已有 C, G，得到 D, H
最终得到 A, B, C, D, E, G, H

所以 BC 是一个超码。

(2) 分解为 3NF

参考答案：

由依赖可推出 C 能决定 A, D, E, G, H，因为：

$C \rightarrow A$
 $C \rightarrow EG$
 $C, G \rightarrow D, H$

因此可分解为：

$R_1(C, A, D, E, G, H)$
 $R_2(B, C)$

其中：

- R_1 中 C 可作为主键；
- R_2 保存候选码 BC，保证无损连接。

解析：

这题的关键是先计算闭包，发现 C 对除 B 外的属性具有很强决定能力。
保留 $R_2(B, C)$ 是为了让原关系的候选码出现在分解结果中。

第五部分：应用题

原题

手机游戏平台关系模式：

```
GAME(Gid, Gname, version, type)
PERSON(Pid, Pname, age, hobby)
PG(Pid, Gid)
```

其中 PG 表示玩家下载的游戏。

1. 分别用关系代数和 SQL 查询玩家“wxy”下载的所有游戏名称

关系代数：

```
 $\pi_{Gname}(\sigma_{Pname='wxy'}(PERSON) \bowtie PG \bowtie GAME)$ 
```

SQL：

```
SELECT g.Gname
FROM PERSON p
JOIN PG pg ON p.Pid = pg.Pid
JOIN GAME g ON pg.Gid = g.Gid
WHERE p.Pname = 'wxy';
```

2. 查询下载数量超过 500 次的游戏名称

关系代数思路：

```
 $\gamma_{Gid, COUNT(Pid) \rightarrow cnt}(PG)$ ，筛选  $cnt > 500$ ，再连接 GAME 投影 Gname
```

SQL：

```
SELECT g.Gname
FROM GAME g
JOIN PG pg ON g.Gid = pg.Gid
GROUP BY g.Gid, g.Gname
HAVING COUNT(*) > 500;
```

3. 查询下载了游戏 G01 但没有下载游戏 G02 的玩家姓名

关系代数：

```
 $\pi_{Pname}(\text{PERSON} \bowtie (\pi_{Pid}(\sigma_{Gid='G01'}(PG)) - \pi_{Pid}(\sigma_{Gid='G02'}(PG)))$ 
```



```
)  
)
```

SQL:

```
SELECT p.Pname  
FROM PERSON p  
WHERE EXISTS (  
    SELECT 1 FROM PG  
    WHERE PG.Pid = p.Pid AND PG.Gid = 'G01'  
)  
AND NOT EXISTS (  
    SELECT 1 FROM PG  
    WHERE PG.Pid = p.Pid AND PG.Gid = 'G02'  
);
```

4. 用关系代数查询下载了所有“益智类”游戏的玩家 id

参考答案:

$$\pi_{Pid, Gid}(PG) \div \pi_{Gid}(\sigma_{type='益智类'}(GAME))$$

解析:

“所有某类游戏”是除法题。被除数是玩家和游戏下载关系，除数是全部益智类游戏编号。

5. 用 SQL 查询年龄 15-25 岁之间的玩家都下载了哪些游戏

参考答案:

```
SELECT DISTINCT g.Gid, g.Gname  
FROM PERSON p  
JOIN PG pg ON p.Pid = pg.Pid  
JOIN GAME g ON pg.Gid = g.Gid  
WHERE p.age BETWEEN 15 AND 25;
```

6. 用户 p01 下载了游戏 G02，应发布什么 SQL?

参考答案:

```
INSERT INTO PG(Pid, Gid)  
VALUES ('p01', 'G02');
```

若防止重复下载记录，可先设置联合主键:

```
ALTER TABLE PG ADD CONSTRAINT PK_PG PRIMARY KEY (Pid, Gid);
```

7. 游戏开发者修改游戏后重新上传，版本号变化，数据库完成什么操作？

参考答案：

```
UPDATE GAME  
    SET version = :new_version  
    WHERE Gid = :gid;
```

如果新版本也要保留历史版本，则应把 version 也纳入主键或建立版本表：

```
GameVersion(Gid, version, upload_time, file_url)
```

解析：

若只保存当前版本，是 UPDATE ；若保存多个版本，是 INSERT 新版本记录。

四、2019-2020 学年第 2 学期《数据库系统原理》考试（A 卷）解析

第一部分：简答题

1. 原题：简述 varchar 与 char 的区别。

参考答案：

- CHAR(n) 是定长字符串，不足长度时通常补空格；
- VARCHAR(n) 是变长字符串，只存实际长度；
- CHAR 适合长度固定的数据，如性别、状态码；
- VARCHAR 适合长度变化较大的数据，如姓名、地址、备注。

2. 原题：什么是数据库索引？

参考答案：

索引是数据库为了提高查询效率而建立的辅助数据结构。它根据某些列的值组织指向数据行的访问路径，类似书的目录。

3. 原题：数据库索引一般采用什么结构？

参考答案：

数据库索引常采用 B+ 树结构。

B+ 树高度低、平衡性好，适合磁盘块访问和范围查询。部分数据库也支持哈希索引、位图索引等。

4. 原题：事务的四种特性指的是什么？

参考答案：

事务四特性即 ACID：

- 原子性；
- 一致性；
- 隔离性；
- 持久性。

5. 原题：在大学数据库中，用 SQL 查询名字中包含 g 的学生的学号、姓名。

参考答案：

```
SELECT ID, name
FROM student
WHERE name LIKE '%g%';
```

6. 原题：简述函数和触发器之间的异同。

参考答案：

相同点：

- 都是数据库中的程序对象；
- 都可以封装逻辑并访问数据库数据。

不同点：

- 函数通常由 SQL 或程序显式调用，并返回一个值；
- 触发器由数据库事件自动触发，如 INSERT、UPDATE、DELETE；
- 函数适合计算和返回结果；
- 触发器适合自动维护规则、日志和派生数据。

7. 原题：distinct 的作用是什么？

参考答案：

DISTINCT 用于去除查询结果中的重复行。

```
SELECT DISTINCT dept_name
FROM student;
```

8. 原题：若关系 R 所有属性都是不可再分的数据项，则该关系最低满足第几范式？

参考答案：

满足第一范式 1NF。

1NF 的基本要求是关系中的每个属性值都是原子的、不可再分的。

9. 原题：简述用户自定义的类型和域之间的差别。

参考答案：

域是属性允许取值的集合或范围，强调取值约束。

用户自定义类型是用户基于系统类型创建的新类型，强调类型定义和类型检查。

例如：

```
CREATE DOMAIN Dollars AS NUMERIC(12,2) CHECK (VALUE >= 0);
```

域可直接带约束；自定义类型更强调抽象数据类型或别名类型。

10. 原题： $AB \rightarrow C$ 能蕴含 $A \rightarrow C$ 、 $B \rightarrow C$ 吗？

参考答案：

不能。

$AB \rightarrow C$ 表示属性组合 A,B 一起才能决定 C，不能推出 A 单独决定 C，也不能推出 B 单独决定 C。

解析：

只有当已知额外依赖，例如 $A \rightarrow B$ 或 $B \rightarrow A$ 时，才可能进一步推出。

第二、三部分：图片中未见可辨认题干

原图片在第一部分简答题之后只出现了“二、”“三、”两个题号标题，未见具体题干内容，随后直接进入第四部分分析题。

因此本解析不补写第二、三部分题目。

第四部分：分析题

1. 原题：在学生表中，将学号 ID 和姓名 name 的组合设计为该表主键是否合理？为什么？

参考答案：

一般不合理。

学号通常已经能唯一标识学生，用 ID 单独作为主键即可。把姓名也放入主键会带来问题：

- 姓名可能重复；
- 姓名可能修改；
- 联合主键变长，外键引用不方便；
- 主键应尽量稳定、简洁、无业务变化风险。

解析：

如果 ID 已唯一，则 (ID, name) 是超码但不是候选码，因为 name 是多余属性。

3. 原题：张三汇款给李四 200 元，张三账户已减 200 美元后系统故障，没有李四账户中增加 200 美元，数据库出现什么状态？如何解决？通过什么手段实现？

参考答案：

数据库处于不一致状态，违反事务的原子性和一致性。

解决方法是把扣款和加款放入同一事务，失败时回滚，成功时提交。数据库通过日志和恢复机制实现。

示例：

```
BEGIN
    UPDATE account
        SET balance = balance - 200
        WHERE name = '张三';

    UPDATE account
        SET balance = balance + 200
        WHERE name = '李四';

    COMMIT;
EXCEPTION
    WHEN OTHERS THEN
        ROLLBACK;
END;
/
```

4. 原题：用 Armstrong 三定律证明合并律。每一步给出依据。

合并律：若 $\alpha \rightarrow \beta$ 且 $\alpha \rightarrow \gamma$ ，则 $\alpha \rightarrow \beta\gamma$ 。

证明：

1. 由 $\alpha \rightarrow \beta$ ，根据增广律，两边加 α ，得：

$\alpha \rightarrow \alpha\beta$

2. 由 $\alpha \rightarrow \gamma$ ，根据增广律，两边加 β ，得：

$$\alpha\beta \rightarrow \beta\gamma$$

3. 由 $\alpha \rightarrow \alpha\beta$ 和 $\alpha\beta \rightarrow \beta\gamma$ ，根据传递律，得：

$$\alpha \rightarrow \beta\gamma$$

证毕。

第五部分：设计题

原题

汽车保险公司管理系统，需要记录：

顾客：顾客ID、名字、家庭住址
汽车：汽车编号、类型、所有顾客ID
事故：事故编号、日期、位置
保险赔付：ID、截止日期、金额、实际赔付日期
保险单：保险单号、保险赔付ID

约束：

- 每位顾客可以有 1 辆或多辆汽车；
- 每辆汽车可关联 0 个或多个事故；
- 每个事故可关联 1 到多辆车；
- 1 个保险单可以覆盖 1 辆或多辆汽车；
- 1 辆车可关联 1 到多个保险单；
- 保险单可关联一个或多个保险赔付。

要求：

1. 设计 E-R 图；
2. 转换为关系模式；
3. 指出每个关系模式的主码。

参考答案：

实体表：

Customer(Customer_ID, Name, Address)
Car(Car_ID, Car_Type, Customer_ID)
Accident(Accident_ID, Accident_Date, Location)

```
Policy(Policy_ID)
Claim(Claim_ID, Deadline, Amount, Actual_Pay_Date)
```

联系表：

```
CarAccident(Car_ID, Accident_ID)
PolicyCar(Policy_ID, Car_ID)
PolicyClaim(Policy_ID, Claim_ID)
```

主码：

- Customer : Customer_ID ;
- Car : Car_ID ;
- Accident : Accident_ID ;
- Policy : Policy_ID ;
- Claim : Claim_ID ;
- CarAccident : (Car_ID, Accident_ID) ;
- PolicyCar : (Policy_ID, Car_ID) ;
- PolicyClaim : (Policy_ID, Claim_ID) 。

解析：

这题的关键是识别多对多联系：

- 汽车和事故是多对多；
- 保险单和汽车是多对多；
- 保险单和赔付是一对多或多对多，按题目“可关联一个或多个”更稳妥地用联系表表达。

第六部分：计算题

原题：关系模式 $R(A, B, C, D, E)$ ，函数依赖集 $F=\{BC \rightarrow AD, AD \rightarrow EB, E \rightarrow C\}$ 。

(1) 判断 R 是否属于 3NF；(2) 判断是否属于 BCNF。

参考答案：

候选码：

```
BC, AD, BE
```

求闭包：

```
(BC)+ = BCAD, AD→EB, 得到 A,B,C,D,E
(AD)+ = ADEB, E→C, 得到 A,B,C,D,E
(BE)+ = BEC, BC→AD, 得到 A,B,C,D,E
```

所有属性 A,B,C,D,E 都是主属性，因为它们都出现在某个候选码中。

判断 3NF：

- $BC \rightarrow AD$ 中 BC 是候选码；
- $AD \rightarrow EB$ 中 AD 是候选码；
- $E \rightarrow C$ 中 E 不是超码，但右部 C 是主属性。

所以 R 满足 3NF。

判断 BCNF：

BCNF 要求每个非平凡依赖的左部都是超码。

$E \rightarrow C$ 中 E 不是超码，因此 R 不满足 BCNF。

第七部分：应用题

原题

银行企业数据库：

```
branch(branch_name, branch_city, assets)
customer(customer_name, customer_street, customer_city)
loan(loan_number, branch_name, amount)
borrower(customer_name, loan_number)
account(account_number, branch_name, balance)
depositor(customer_name, account_number)
```

带下划线属性为主码，假设客户名字不相同。

1. 用 SQL 创建表

参考答案示例：

```
CREATE TABLE branch (
    branch_name VARCHAR2(40) PRIMARY KEY,
    branch_city VARCHAR2(40),
    assets NUMBER(12,2)
);

CREATE TABLE customer (
    customer_name VARCHAR2(40) PRIMARY KEY,
    customer_street VARCHAR2(80),
    customer_city VARCHAR2(40)
);

CREATE TABLE loan (
    loan_number VARCHAR2(20) PRIMARY KEY,
    branch_name VARCHAR2(40) NOT NULL,
```



```

    amount NUMBER(12,2),
    CONSTRAINT FK_LOAN_BRANCH
        FOREIGN KEY (branch_name) REFERENCES branch(branch_name)
);

CREATE TABLE borrower (
    customer_name VARCHAR2(40),
    loan_number VARCHAR2(20),
    PRIMARY KEY (customer_name, loan_number),
    FOREIGN KEY (customer_name) REFERENCES customer(customer_name),
    FOREIGN KEY (loan_number) REFERENCES loan(loan_number)
);

CREATE TABLE account (
    account_number VARCHAR2(20) PRIMARY KEY,
    branch_name VARCHAR2(40) NOT NULL,
    balance NUMBER(12,2),
    FOREIGN KEY (branch_name) REFERENCES branch(branch_name)
);

CREATE TABLE depositor (
    customer_name VARCHAR2(40),
    account_number VARCHAR2(20),
    PRIMARY KEY (customer_name, account_number),
    FOREIGN KEY (customer_name) REFERENCES customer(customer_name),
    FOREIGN KEY (account_number) REFERENCES account(account_number)
);

```

2. 用关系代数和 SQL 找出在 Brighton 银行中有存款的所有客户姓名、存款号和存款额

关系代数：

```

 $\pi_{\text{customer\_name}, \text{account\_number}, \text{balance}}($ 
     $\text{depositor} \bowtie \text{account} \bowtie \sigma_{\text{branch\_name} = \text{'Brighton'}}(\text{branch})$ 
 $)$ 

```

SQL：

```

SELECT d.customer_name, a.account_number, a.balance
FROM depositor d
JOIN account a ON d.account_number = a.account_number
WHERE a.branch_name = 'Brighton';

```

3. 找出账户平均余额小于 5000 元的支行，显示支行名称和账户平均余额

SQL：

```
SELECT branch_name, AVG(balance) AS avg_balance
FROM account
GROUP BY branch_name
HAVING AVG(balance) < 5000;
```

关系代数可用扩展关系代数聚合表达：

```
 $\sigma_{avg\_balance < 5000}(\gamma_{branch\_name, AVG(balance) \rightarrow avg\_balance}(account))$ 
```

4. 找出所有在银行中有贷款但无账户的客户

SQL：

```
SELECT DISTINCT b.customer_name
FROM borrower b
WHERE NOT EXISTS (
    SELECT 1
    FROM depositor d
    WHERE d.customer_name = b.customer_name
);
```

关系代数：

```
 $\pi_{customer\_name}(borrower) - \pi_{customer\_name}(depositor)$ 
```

5. 对所有存款余额大于平均存款额的账户付 3% 利息

SQL：

```
UPDATE account
SET balance = balance * 1.03
WHERE balance > (
    SELECT AVG(balance)
    FROM account
);
```

解析：

若数据库不允许在同一更新语句中直接引用被更新表的聚合结果，可先把平均值保存到变量或临时表中。

五、2020-2021 学年第 1 学期 2018 级《数据库系统原理》考试试题 (A 卷) 解析

第一部分：简答题

1. 原题：DDL 和 DML 的含义分别是什么？

参考答案：

- DDL：数据定义语言，用于定义数据库结构，如 CREATE、ALTER、DROP；
- DML：数据操纵语言，用于查询和修改数据，如 SELECT、INSERT、UPDATE、DELETE。

2. 原题：在 from 子句中：from R, (子查询)，子查询中可以直接使用关系 R 吗？为什么？

参考答案：

一般不能直接使用同一层 FROM 中前面出现的关系 R，因为普通派生表子查询在逻辑上先独立形成结果，再与 R 做连接。

如果要引用外层关系，应使用相关子查询或支持 LATERAL / APPLY 的语法。

示例：

```
FROM R, LATERAL (SELECT ... WHERE ... = R.a) X
```

3. 原题：请罗列 3 种数据模型。

参考答案：

常见数据模型包括：

- 层次模型；
- 网状模型；
- 关系模型；
- 面向对象模型；
- 半结构化模型。

写出其中三种即可。

4. 原题：在 ER 图转化关系模式时，如何处理联系集？

参考答案：

处理方法取决于联系基数：

- 1:1 联系：可在任一实体表中加入对方主键作为外键；

- 1:n 联系：在 n 端实体表中加入 1 端主键作为外键；
- m:n 联系：转换成独立关系表，包含各参与实体的主键及联系自身属性；
- 多元联系：通常转换成独立关系表。

5. 原题：为什么 3NF 范式分解能够保证函数依赖一定能够得到保持？

参考答案：

3NF 综合算法会根据最小函数依赖集为每个依赖建立关系模式，并在必要时加入包含候选码的关系模式。

因此原函数依赖集中的每个依赖都能在某个分解后的关系中体现，从而保证依赖保持。

6. 原题：请简述函数和过程的异同。

参考答案：

相同点：

- 都是数据库程序单元；
- 都可以有参数；
- 都可以封装业务逻辑。

不同点：

- 函数必须返回一个值，可用于表达式或查询；
- 过程不要求返回值，常通过输出参数返回结果；
- 过程更适合完成一段业务流程；
- 函数更适合计算单个结果。

7. 原题：用属性集合闭包表达属性集合 A 是关系 R 的一个候选码。

参考答案：

属性集 A 是候选码，当且仅当：

$$A^+ = R$$

且对 A 的任意真子集 B，都有：

$$B^+ \neq R$$

即 A 能推出全部属性，并且是最小的。

8. 原题：如果两个关系进行集合并运算，那么这两个关系需要满足哪两个条件？

参考答案：

集合并运算要求两个关系并相容：

- 属性个数相同；
- 对应属性的域或数据类型相同或兼容。

9. 原题：在 SQL 中，WHERE 子句可以嵌入子查询。该子查询的作用有哪些？至少两种。

参考答案：

子查询可用于：

- 集合成员判断：IN、NOT IN；
- 存在性判断：EXISTS、NOT EXISTS；
- 比较判断：> (SELECT AVG(...))；
- 集合比较：> ALL、< ANY；
- 相关查询，实现逐行条件判断。

10. 原题：布尔表达式 $(1 > \text{NULL}) \wedge (\text{NULL} = \text{NULL})$ 的结果是什么？

参考答案：

结果是 UNKNOWN。

解析：

在 SQL 三值逻辑中，任何与 NULL 的普通比较结果通常都是 UNKNOWN：

```
1 > NULL          → UNKNOWN
NULL = NULL       → UNKNOWN
UNKNOWN AND UNKNOWN → UNKNOWN
```

第二部分：分析题

1. 原题：证明分解律：若有函数依赖 $\alpha \rightarrow \beta\gamma$ 成立，则有 $\alpha \rightarrow \beta$ 、 $\alpha \rightarrow \gamma$ 。

证明：

由自反律可知：

```
 $\beta\gamma \rightarrow \beta$ 
 $\beta\gamma \rightarrow \gamma$ 
```

又已知：

```
 $\alpha \rightarrow \beta\gamma$ 
```

根据传递律：

$$\alpha \rightarrow \beta$$
$$\alpha \rightarrow \gamma$$

证毕。

2. 原题：给定调度 s ，判断是否为冲突可串行化调度。

图中调度：

```
1  T1: W(A)
2  T2: W(A)
3  T2: R(B)
4  T1: W(B)
```

参考答案：

对数据项 A：

T1: W(A) 在 T2: W(A) 之前，因此有 $T1 \rightarrow T2$

对数据项 B：

T2: R(B) 在 T1: W(B) 之前，因此有 $T2 \rightarrow T1$

优先图存在环：

$$T1 \rightarrow T2 \rightarrow T1$$

因此该调度不是冲突可串行化调度。

3. 原题：查询计算机学院学生选课信息，有两种关系代数表达式，分析优劣：

- ① $\sigma_{dept_name='Comp.Sci.'}(student \bowtie takes)$
- ② $(\sigma_{dept_name='Comp.Sci.'}(student)) \bowtie takes$

参考答案：

第二种更优。

因为它先在 `student` 表中筛选计算机学院学生，再与 `takes` 连接，可以减少参与连接的元组数量，从而降低连接代价。

解析：

这是关系代数查询优化中的“选择下推”。
等价变换思路是尽量把选择操作提前执行。

4. 原题：构造 E-R 图时，有些实体集自身属性不足以构成主码，应如何处理？ 转化为关系模式时属性集包括哪些内容？

参考答案：

这种实体集应建模为弱实体集。

弱实体依赖强实体存在，使用强实体主码加强实体自身的部分键共同标识。

转换成关系模式时，弱实体关系应包括：

- 弱实体自身属性；
- 弱实体的部分键；
- 所依赖强实体的主键；
- 必要时包括标识联系的属性。

其主键通常为：

强实体主键 + 弱实体部分键

第三部分：设计题

原题

吉林大学图书馆欲开发管理系统。数据库需要保存：

图书：图书编号、书名、价格、作者（图书馆中可能有区分同一种书）
书库：书库号、地点、面积、电话
管理员：工号、姓名、性别、职务
学生：学号、姓名、性别、年龄、学院

约束：

- 一名学生可以借 0 到 5 本图书；
- 借书过程中需要记录借阅起止时间；
- 书库中可以存放图书，需要记录存放每种书的数量；
- 一名管理员负责管理一个书库。

要求：

1. 设计 E-R 图；
2. 转换为关系模式；
3. 指出每个关系模式的主码。

参考答案：

实体表：

```
Book(Book_ID, Book_Name, Price, Author)
LibraryRoom(Room_ID, Location, Area, Phone)
Manager(Manager_ID, Manager_Name, Sex, Position, Room_ID)
Student(Sid, Sname, Sex, Age, College)
```

联系表：

```
Borrow(Sid, Book_ID, Borrow_Start, Borrow_End)
Store(Room_ID, Book_ID, Quantity)
```

主码：

- Book：Book_ID；
- LibraryRoom：Room_ID；
- Manager：Manager_ID；
- Student：Sid；
- Borrow：可用 (Sid, Book_ID, Borrow_Start)；
- Store：(Room_ID, Book_ID)。

完整性约束：

- Manager.Room_ID 外键引用 LibraryRoom，并可加唯一约束表示一个管理员管理一个书库；
- Borrow.Sid 引用 Student；
- Borrow.Book_ID 引用 Book；
- Store.Room_ID 引用 LibraryRoom；
- Store.Book_ID 引用 Book；
- Quantity >= 0；
- 学生借书数量不超过 5 本可通过触发器、断言或应用逻辑控制。

解析：

代表性方法：

- “学生借书”是学生和图书之间的多对多联系，且联系有起止时间属性；
- “书库存放图书”也是多对多联系，且联系有数量属性；
- “管理员管理书库”是一对一或多对一联系，按题意一名管理员负责一个书库，可在管理员表中放 Room_ID。

第四部分：计算题

原题

关系模式 $R<U, F>$, $U=\{A, B, C, D, E, F\}$, 函数依赖集:

$F=\{A\rightarrow BCD, BC\rightarrow DE, B\rightarrow D, D\rightarrow A\}$

1. 证明 DF 是一个候选码;
2. 写出所有候选码。

1. 证明 DF 是候选码

参考答案:

计算闭包:

$(DF)^+$

初始: D, F

$D\rightarrow A$, 得到 A

$A\rightarrow BCD$, 得到 B, C, D

$BC\rightarrow DE$, 得到 E

最终得到 A, B, C, D, E, F

因此:

$(DF)^+ = U$

说明 DF 是超码。

再看最小性:

- $D^+=\{D, A, B, C, E\}$, 不能推出 F ;
- $F^+=\{F\}$, 不能推出其他属性。

所以 DF 是候选码。

2. 写出所有候选码

参考答案:

所有候选码为:

AF, BF, DF

解析:

属性 F 不出现在任何函数依赖右部, 因此每个候选码都必须包含 F 。

又有:

```
A+ = {A,B,C,D,E}
B+ = {A,B,C,D,E}
D+ = {A,B,C,D,E}
```

所以加上 F 后均能推出全部属性：

```
AF, BF, DF
```

C、E 不能单独推出 A,B,D 等关键属性，所以 CF、EF 不是候选码。

第五部分：应用题

原题

大学数据库关系模式如下：

```
department(dept_name, building, budget)
instructor(ID, name, dept_name, salary)
course(course_id, title, dept_name, credits)
section(course_id, sec_id, semester, year, building, room_number,
time_slot_id)
teaches(ID, course_id, section_id, semester, year)
student(ID, name, dept_name, tot_cred)
prereq(course_id, prereq_id)
Advisor(s_id, i_id)
takes(ID, course_id, sec_id, semester, year, grade)
classroom(building, room_number, capacity)
time_slot(time_slot_id, day, start_time, end_time)
```

使用关系代数和 SQL 完成查询。

1. 查询在 2020 年秋季没有选课的学生 ID

关系代数：

```
 $\pi_{ID}(student) - \pi_{ID}(\sigma_{year=2020 \wedge semester='Fall'}(takes))$ 
```

SQL：

```
SELECT s.ID
FROM student s
WHERE NOT EXISTS (
    SELECT 1
    FROM takes t
    WHERE t.ID = s.ID
    AND t.year = 2020
)
```

```
        AND t.semester = 'Fall'
    );
```

2. 查询选了名叫 Einstein 的老师开设的所有课程的学生姓名，教师和学生均无重名

参考答案：

```
SELECT DISTINCT s.name
FROM student s
WHERE NOT EXISTS (
    SELECT *
    FROM instructor i
    JOIN teaches te ON i.ID = te.ID
    WHERE i.name = 'Einstein'
    AND NOT EXISTS (
        SELECT *
        FROM takes ta
        WHERE ta.ID = s.ID
            AND ta.course_id = te.course_id
            AND ta.sec_id = te.section_id
            AND ta.semester = te.semester
            AND ta.year = te.year
    )
);
```

关系代数思路：

先求 Einstein 开设的课程段集合 E；
再用 $\text{takes} \div E$ 得到选了全部这些课程段的学生 ID；
最后连接 student 投影姓名。

解析：

这题是 SQL 中的“除法”或“所有”问题，经典写法是双重 NOT EXISTS。

3. 查询在 2020 年秋季开课，但不在 2021 年春季开课的所有课程 ID

关系代数：

```
 $\pi_{\text{course\_id}}(\sigma_{\text{year}=2020 \wedge \text{semester}='Fall'}(\text{section}))$   
-  
 $\pi_{\text{course\_id}}(\sigma_{\text{year}=2021 \wedge \text{semester}='Spring'}(\text{section}))$ 
```

SQL：

```

SELECT DISTINCT course_id
FROM section
WHERE year = 2020
      AND semester = 'Fall'
MINUS
SELECT DISTINCT course_id
FROM section
WHERE year = 2021
      AND semester = 'Spring';

```

MySQL 可写为:

```

SELECT DISTINCT s1.course_id
FROM section s1
WHERE s1.year = 2020
      AND s1.semester = 'Fall'
      AND NOT EXISTS (
        SELECT 1
        FROM section s2
        WHERE s2.course_id = s1.course_id
              AND s2.year = 2021
              AND s2.semester = 'Spring'
      );

```

4. 给年薪大于 7 万元的老师涨 5% 工资，给年薪小于等于 7 万元的老师涨 10% 工资

参考答案:

```

UPDATE instructor
SET salary =
CASE
    WHEN salary > 70000 THEN salary * 1.05
    ELSE salary * 1.10
END;

```

解析:

必须用一条 UPDATE + CASE，避免先涨低工资再涨高工资造成重复更新。

5. 不使用聚合函数，查询老师的最高工资

参考答案:

```

SELECT DISTINCT salary
FROM instructor i
WHERE NOT EXISTS (

```

```
SELECT 1
FROM instructor j
WHERE j.salary > i.salary
);
```

解析：

不使用 MAX 时，可以用“找不存在比它更大的工资”的思路。
若有多个老师同为最高工资，DISTINCT 可以只返回一个工资值。

复习建议

这五套题的共同特点是：概念题覆盖面广，应用题反复考“关系代数 + SQL”，计算题集中在函数依赖、候选码、范式分解和调度可串行化。

优先掌握以下模板：

- 名词解释：定义 + 作用 + 示例；
- E-R 设计：实体、联系、基数、关系模式、主键外键；
- 候选码：先找不在右部的属性，再求闭包；
- 3NF/BCNF：先求候选码和主属性，再逐条判断依赖；
- 调度题：画优先图，找环；
- 除法题：关系代数用 $\div$ ，SQL 用双重 NOT EXISTS；
- 事务题：原子性、一致性、日志、回滚、提交；
- 并发题：锁、两阶段锁、死锁、条件更新。